USTHB
Module ASD3

Faculty of Computer Science Year
2025/2026

Implementation of a Game using Dynamic Data Structures

Team members:

Yasmine Zitouni G4

Aya Bouchra Amir G4

INTRODUCTION:

Data structures play a fundamental role in the design and implementation of efficient computer programs. Many real-world applications rely on dynamic linear structures such as lists and queues to manage data whose size and order change during execution. Mastering these structures is therefore essential for understanding how complex systems are built and organized.

This project, aims to apply theoretical concepts related to dynamic data structures through the implementation of a game simulation. The game is based on a set of players managed using queues and linked lists, where each player participates in successive rounds according to predefined rules and priorities. The evolution of the game depends on random values generated by the system and on the players' performance across different rounds.

The project is divided into two main parts. The first part implements an initial game strategy based on digit-sum evaluation, while the second part introduces a modified strategy using the greatest common divisor (GCD) of randomly generated numbers. Throughout the game, players are dynamically moved between different queues and lists according to their results, until the game reaches its stopping conditions.

This report presents the algorithmic conception of the solution, the data structures used, the decomposition of the program into functions, and an analysis of the complexity of the main operations. It also discusses the implementation choices made in the C programming language and provides an overview of the obtained results.

Objectives Of The Work:

The main objective of this project is to apply the theoretical concepts studied in the Algorithmics module through the design and implementation of a complete program using dynamic data structures.

More specifically, the objectives of this work are as follows:

- To design and implement dynamic linear data structures such as queues and linked lists using the C programming language.
- To manipulate chained data structures through fundamental operations including insertion, deletion, traversal, and ordered insertion.
- To decompose a complex problem into smaller and well-defined functions and procedures in order to obtain a clear and modular program structure.
- To simulate a game scenario involving multiple players, where the evolution of the game depends on random values and predefined rules.
- To manage players dynamically by moving them between different queues and lists according to their performance during the game.
- To analyze and evaluate the theoretical complexity of the main operations used in the program.
- To compare two different game strategies and observe their impact on the progression and fairness of the game.

Through this project, the aim is also to strengthen practical programming skills, improve problem-solving abilities, and gain experience in writing a structured technical report.

The Algorithmic Conception:

1\ Description of the used Data types in the game :

```
// Player structure stores individual player information

type Player = Record
```

```
   Num, age : integer ;         // Player ID & Player Age
name : String[50];      // Player name

   Wins, losses, score : integer ;        // Total wins & Total losses + Score
consecutiveWins : integer ; // For tracking streak bonuses

ENDRecord;
```

```
// Basic node for linked list/queue implementation
type Element = Record

   info : Player;          /* Player data */      next : ^Element;        // Pointer to next node
ENDRecord; type
Queue = Record

   Head , Tail : ^Element;
ENDRecord;
```

```
type Match = Record // Match structure stores game round history
gameround : integer;     // Round number

   RoundName, Strategy : String[20];  // "Round: X" "Digit Sum" or "GCD"
player1Num , player1Score, player1Wins, player1Losses : integer;
player1Name : String[50];     player2Name : String[50];

   Player2Num, player2Score, player2Wins, player2Losses : integer;                    ENDRecord;
```

```
// Node for match history linked list
type MatchNode = Record

   matchInfo : Match;      /* Match data  */         next : ^MatchNode;      // Next match in history
```

```
ENDRecord;

//  list for storing match

history type MatchList =

Record      head , tail :

^MatchNode;


ENDRecord;
```

2\ GAME Core Functions for Part1 :

We decided to decompose the solution into the following functions, and same for 2nd part of the game :

```
// Generate random 6-digit number: O(1)

function GenerateRandomNbr(): Integer


begin return (GenerateRandomNbr <-- 100000 + Random(900000));                    end;
```

```
// Calculate sum of digits: O(d) where d = number of digits (max 6)

function SumDigits (In/ n : integer): integer                    Var sum : integer ;

Begin         sum <-- 0 ; While (n <> 0) do sum <-- sum + (n mod 10) ; n <-- n div 10 ;

Endwhile;                return sum ;            End ;
```

```
function CheckWinCond1 (In/ sum : integer) : boolean
// Check if sum is multiple of 5: O(1)

Begin      If (sum mod 5 = 0) then return true ; else return false ;                    Endif;                End
;
```

```
// Play one turn in Part 1: O(d) where d = digits in random number

function playTurnP1 (in/out nbr : integer , in/out sum : integer) :

integer  Begin nbr <-- GenerateRandomNbr () ; // O(1)  sum <--

SumDigits (nbr) ; // O(d)


 If (CheckWinCond1 (sum)) then  /* O(1) */          return 1 ; // win*/      else  return 0 ; // lose

Endif; End ;
```

```
// Play complete round in Part 1: O(12 * d) = O(12 * 6) = O(72) = O(1)

function PlayRoundP1 (In/out p1 : *Player , In/out p2 : *Player, In/out score1 : integer , In/out
score2 : integer) : integer

Var          turns , nbr1 , nbr2, sum1 , sum2 : integer;

Begin score1 <-- 0 ; score2 <-- 0 ; turns <-- 0 ;

// Stop condition 12 turns or until 3 point difference while
((abs(score1 - score2) < 3) and (turns < 12)) do      write
(" Turn : " , turns+1);     score1 <-- score1 + playTurnP1
(nbr1 , sum1) ;  // O(d)
    write(p1.name , " generated number : " , nbr1 , " sum : " , sum1 ,
"score : " , score1);
    score2 <-- score2 + playTurnP1 (nbr2 , sum2) ;  // O(d)
    write(p2.name , " generated number : " , nbr2 , " sum : " , sum2 ,
"score : " , score2);
turns <-- turns + 1 ;
Endwhile;


// Update player statistics: O(1)
```

```
if (score1 > score2) then
    p1^.wins <-- p1^.wins + 1 ;
    p1^.score <-- p1^.score + score1 ;
    p2^.losses <-- p2^.losses + 1 ;
    p2^.score <-- p2^.score + score2 ;
    return 1 ; // p1 wins
else if (score2 > score1) then
    p2^.wins <-- p2^.wins + 1 ;
    p2^.score <-- p2^.score + score2 ;
    p1^.losses <-- p1^.losses + 1 ;
    p1^.score <-- p1^.score + score1 ;
    return 2 ; // p2 wins
else
    return 0 ; // draw
Endif;

End ;
```

Placing players according to part1 rules :

```
// Place players after Part 1 round: O(1) + O(n) for list insertion procedure

PlacePlayersPart1(in/out F, F1, F3: Queue; in/out LG, LP: ^Element;      in/ result:

integer; in/ p1, p2: Player); begin     if (result = 1) then  // p1 wins     // winner

handling        if (p1.wins >= 5) then   InsertSorted(LG, p1);  // O(n) - needs to find

position        else if (p1.consecutiveWins >= 3) then   Enqueue(F1, p1);      // O(1)

endif


      // loser handling        if (p2.losses >= 5) then              Insert(LP, p2);       // O(n) - append to
end        else if (p2.losses >= 3) the       Enqueue(F3, p2);      // O(1)   else   Enqueue(F, p2);
// O(1)        endif;           // same thing for p2 wins and draw cases                  end;
```

3\ GAME Core Functions for Part2 :

```
// Calculate GCD using Euclidean algorithm: O(log(min(a,b))) function
gcd(a, b: integer): integer; begin     while(b ≠ 0) do        temp ← b;   b ← a
mod b;      a ← temp;       endwhile;       return a;           end;
```

```
// Check if digit exists in number: O(d) where d = digits in
num function digitInNumber(digit, num: integer): boolean;
begin     if (digit = 0 and num = 0) then        return true;
while( num > 0) do

      if (num mod 10 = digit) then   return true;      num ← num div 10;    endwhile;
   Else return false;             end;
```

```
      digit ← temp mod 10;        if (digitInNumber(digit, nbra) or
digitInNumber(digit, nbrb)) then
```

```
// Play one turn in Part 2: O(log(min(a,b)) + d1 + d2)//same logic like part1
// where d1 = digits in gcd, d2 = digits in random numbers
function playTurnP2(In/out nbra : integer, In/out nbrb : integer,  In/out g : integer): integer;
var digit, temp: integer;
begin
  nbra ← GenerateRandomNbr() ; // O(1)    nbrb ← GenerateRandomNbr() ; // O(1)
  g ← gcd(nbra, nbrb);      */O(log(min(a,b))) */          temp ← g;
  if (temp = 0) then
     return 0 ; // gcd = 0
  repeat
        return 1;  endif;   temp ← temp div 10;  until temp = 0;  /*O(d) */   return 0;            end;
```

For player round in part2 has the same logic like part1 as well as placing players after a round in part2 there's a specific function similar to the one of part1 with the different rules

Player selection for rounds :

```
function SelectNextPlayer(In/out F, F1, F3: Queue): Player;

var

  P , tempPlayer: Player;  /*For temporary storage*/tempQueue: Queue;    // For reenqueue
operations

begin

  // Priority 1 F1 (if not empty)

  if not EmptyQueue(F1) then // Check if only one player remains in F1

    if (F1.Head = F1.Tail) then     // Only one in F1, need one from F (if available)

      if not EmptyQueue(F) then   // Take player from F

        Dequeue(F, P);   return P;

      Else   // F is empty, take from F1 anyway

        Dequeue(F1, P);  return P;        Endif;

    Else     // More than one in F1, take from F1 normally

      Dequeue(F1, P);      return P;        Endif;

  Endif;


  // Priority 2 F

  if not EmptyQueue(F) then      // Check if only one player remains in F

    if (F.Head = F.Tail) then        // Only one in F, need one from F3 (if available)
```

```
      if not EmptyQueue(F3) then        // Take player from F3 (not from F!)
        Dequeue(F3, P);            // Check if F3 is now empty (was the last player)
        if EmptyQueue(F3) then
          P.losses <-- P.losses + 1;  // Last player loses
        Endif;                return P;
      else            // F3 is empty, take from F anyway
        Dequeue(F, P);           return P;       Endif;
    else       // More than one in F, take from F normally
      Dequeue(F, P);         return P;     Endif;   Endif;

  // Priority 3  F3
  if not EmptyQueue(F3) then      // Check if only one player remains in F3
    if (F3.Head = F3.Tail) then        // Last player in F3 - he loses
      Dequeue(F3, P);         P.losses <-- P.losses + 1;         return P;
    else        // More than one in F3, take normally
      Dequeue(F3, P);
      // Check if this leaves only one player in F3
      if (F3.Head = F3.Tail) then
        // The remaining player will lose when selected next time
      Endif;         return P;      Endif;
  Endif;

  // All queues empty  create and return a null player
  P.num <-- -1;   P.name <-- 'NO_PLAYER';   return P;       End;
```

Summarizing all functions implemented before going to the main algorithm :

1. Basic Queue Operations:

- `InitQueue(F)`: Initializes an empty queue
- `Enqueue(F, player)`: Adds a player to the end of a queue
- `Dequeue(F, player)`: Removes a player from the front of a queue
- `EmptyQueue(F)`: Checks if a queue is empty
- `HeadQueue(F)`: Returns the front player without removal
- `TailQueue(F)`: Returns the rear player without removal • `QueuesEmpty(F, F1, F3)`: Checks if all queues are empty

2. Player Management:

- `CreateInitialPlayers(F, numPlayers)`: Automatically generates initial player queue
- `SelectNextPlayer(F, F1, F3)`: Selects next player based on priority rules
- `Insert(L, player)`: Adds player to a linked list (unsorted)
- `InsertSorted(L, player)`: Adds player to list in descending score order
- `ConsecutiveWins(result, p1, p2)`: Updates consecutive win streaks

3. Part 1 of the Game (Digit Sum Strategy):

- `GenerateRandomNbr()`: Generates random 6-digit number
- `SumDigits(n)`: Calculates sum of digits in a number
- `CheckWinCond1(sum)`: Checks if sum is multiple of 5
- `playTurnP1(nbr, sum)`: Executes one turn in Part 1
- `PlayRoundP1(p1, p2, score1, score2)`: Manages complete Part 1 round • `PlacePlayersPart1(F, F1, F3, LG, LP, result, p1, p2)`:

  Places players after Part 1 round

4. Part 2 of the Game(GCD Strategy):

- `gcd(a, b)`: Calculates greatest common divisor
- `digitInNumber(digit, num)`: Checks if digit exists in number
- `playTurnP2(nbra, nbrb, g)`: Executes one turn in Part 2

- `PlayRoundP2(p1, p2, score1, score2)`: Manages complete Part 2 round • `PlacePlayersPart2(F, F1, F3, LG, LP, result, p1, p2)`:
  Places players after Part 2 round

5. Game History and Display:

- `SaveGame(ML, result, round, p1, p2, score1, score2, strategy)`: Saves match history
- `DisplayRoundResult(round, result, score1, score2, p1, p2)`: Displays round results
- `DisplayWinners(LG)`: Displays top 3 winners
- `DisplayQueueStatus(F, F1, F3)`: Shows queue player counts
- `DisplayLists(LG, LP)`: Displays winner and loser lists
- `InitMatchList(ML)`: Initializes match history list

6. Time Functions:

- `GetCurrentTime(timeStr)`: Gets current timestamp • `CalculateDuration(startTime, endTime, durationStr)`:
  Calculates time duration
- `ParseTime(timeStr, hour, minute, second)`: Parses time string

7. The After Game functions : • `moveQueueToList(Q, L)`: Moves all players

from queue to list

- `moveQueueToSortedList(F1, LG)`: Moves F1 players to sorted winners list

Main Algorithm :

```
Algorithm Main_Game
var
   F,F1,F3: Queue;
   LG,LP: ^Element;
   numPlayer: integer;
   totalMatchesPart1, totalMatchesPart2: integer;
   matchNumber: integer;
   p1, p2: Player;
   result: integer;
   score1, score2 : integer;
   ML : MatchList;
   roundNbr : integer;
begin
   // Initialization: O(1)
   LG ← NULL; LP ← NULL;
   InitRandom();
   InitQueue(F); InitQueue(F1); InitQueue(F3);
   InitMatchList(ML);


   // Create initial players: O(n) where n = numPlayer
   numPlayer ← 4; //to test
   CreateInitialPlayers(F, numPlayer);
 totalMatchesPart1 ← 0;     totalMatchesPart2 ← 0;
 // Get first two players
```

```
   Dequeue(F, p1);  // O(1)
   Dequeue(F, p2);  // O(1)

   // PART 1 MAIN LOOP: O(3n * round_complexity)
write("=== GAME PART 1 START ===");
RoundNbr<-- 1;

   while (not QueuesEmpty(F, F1, F3) && totalMatchesPart1 < 3*numPlayer)
do        write("--- Round", roundNbr, "---");        // Play round: O(1) constant
time per round        result = PlayRoundP1(p1, p2, score1, score2, roundNbr);
// Update and place players: O(n) worst case for list insertion
consecutiveWins(result, p1, p2);

      SaveGame(ML, result, totalMatchesPart1, p1, p2, score1, score2, 1);
      DisplayRoundResult(totalMatchesPart1+1 ,result ,score1 ,score2 ,p1,
p2);        PlacePlayersPart1(F, F1, F2, LG, LP, result, p1, p2);


      // Select next players: O(1)        if
result = 1 then            p2 =
SelectNextPlayer(F, F1, F2);        else
if result = 2 then            p1 =
SelectNextPlayer(F, F1, F2);        else

         p1 = SelectNextPlayer(F, F1, F2);
p2 = SelectNextPlayer(F, F1, F2);
endif;
```

```
      // Display status: O(n) to traverse all queues/lists
      DisplayQueueStatus(F, F1, F2);
DisplayLists(LG, LP);

      totalMatchesPart1++;      roundNbr++;    Endwhile;   write("===  GAME
PART 1 ENDED  ==="); // same complexity evaluation as part1    if (not
QueuesEmpty(F, F1, F2) && totalMatchesPart1 >= 3*numPlayer) then
write("==== STRATEGY CHANGE ====");      write("===  GAME PART 2 START
===");


     roundNbr <--1;


   while (not QueuesEmpty(F, F1, F2) && totalMatchesPart2 < 2*numPlayer)
do       result = playRoundP2(p1, p2, score1, score2, roundNbr);
consecutiveWins(result, p1, p2);

      SaveGame(ML, result, totalMatchesPart1, p1, p2, score1, score2, 2);
      DisplayRoundResult(roundNbr,result ,score1 ,score2 ,p1,
p2)       PlacePlayersPart2(F, F1, F2, LG, LP, result, p1, p2);
p1 = SelectNextPlayer(F, F1, F2);       p2 = SelectNextPlayer(F,
F1, F2);

     DisplayQueueStatus(F, F1, F2);
DisplayLists(LG, LP);
totalMatchesPart2 ++;
```

```
    roundNbr++;
  Endwhile;

  // Check Part 2 game end condition
  if (totalMatchesPart2 ≥ 2 * numPlayer) then
    write("=== GAME PART 2 ENDED ===");
    // Move remaining players
    moveQueueToSortedList(F1, LG);// LG must be sorted
    moveQueueToList(F, LP);
    moveQueueToList(F3, LP);
  ENDif;

  // displaying the winners
  DisplayWinners(LG);
  write("==== Game End! ====");
End;
```

CONCLUSION :

Looking Back: What We Built and Learned

This project has been an exciting journey from theoretical concepts to practical implementation. We started with simple data structures on paper and ended up creating a complete, dynamic game simulation. The experience taught us that queues and linked lists are more than just simple concepts ,they're powerful tools for modeling real world systems like games where order, priority, and dynamic movement matter.

The biggest challenge wasn't just writing the code, but designing a system where players naturally flow between queues based on their performance. Watching a player start in the main queue, move to F1 after three consecutive wins, then possibly end up in the winners' list (LG) felt rewarding it proved our structures were working as intended.

We also learned the importance of planning before coding, especially in this kind of complex project. The time spent designing clear functions like SelectNextPlayer() and PlacePlayersPart1/2() saved us hours of debugging later. In the beginning, the project looked so hard to even understand but breaking it into small, testable pieces made it manageable.

Complexity and Performance Reflections:

- **Queue operations (Enqueue/Dequeue) are beautifully efficient at O(1)**, making them perfect for managing players in real-time.
- **Sorted insertion in LG takes O(n)**, which is acceptable since the winners' list stays relatively small.

- **The game loop scales well**—even with more players, the per-match operations remain constant.

Fairness in the Game:

After observing many simulations, we believe that our game is fair and achieve a perfect ballence between skill and chance .

Elements of fairness:

- **The priority system (F1 → F → F3)**, it is as a reward for the consistent winners without making them unbeatable.
- **Multiple pathways to success** ,winning streaks or total wins allow different player types to succeed.
- **The mid game rule change** resets strategies and gives trailing players another chance.
- **The role of randomness:**
  While luck plays a significant role (through random number generation), the game's structure ensures that consistent performance gets rewarded over time. A lucky newcomer might win a match, but only consistently good players reach the winners' list.

Finally:

This project started as an algorithm assignment but became a lesson in system design, problem decomposition, and creative implementation. We didn't just implement data structures we gave them purpose in a living, simulation.

The most satisfying moment wasn't when the code compiled without errors, but when we ran the simulation and saw emergent behavior: players rising and falling, queues emptying and filling... That's the magic of programming creating systems that come to life.

As we submit this project, we carry forward not just better coding skills, but a deeper appreciation for how simple structures, thoughtfully combined, can create complex and engaging systems .